



جامعة أم القرى

UMM AL-QURA UNIVERSITY

Stage 5: Final Report

Stress Detection Using Facial Recognitions

Supervised By Dr. Seereen Noorwali

Group 1

Task Distribution

Name	ID	Group	Task
Retaj Alhazmi	445006594	2	- Proposed Methodology & Implementation
Manar Ahmed	445006780	1	- Experimental Results and Discussion
Ghala Shahir	445006326	2	- Introduction - Related Work
Ghadeer Sami	44511798	2	- Experimental Setup
Rawan Mohammed	445013919	2	- Identified Limitations - Conclusion and Future Work



Table of Contents

1 . Introduction.....	2
1.1 Background and Motivation	2
1.2 Problem Statement.....	3
1.3 Research Objectives.....	3
2. Related Work.....	4
2.1 Research Gap	6
2.2 DataSets	7
3 . Proposed Methodology & Implementation.....	11
3.1 CNN+LSTM Experiment	11
3.2 MobileNetV2 + GRU (kaggle) Experiment.....	12
3.3 Real-Time VGG16 Colab Experiment.....	13
3.4 MATLAB-Based VGG16 Experiment.....	15
3.5 Final VGG16-Based Solution	17
4. Experimental Setup	21
5. Experimental Results and Discussion	23
6. Identified Limitation	25
7. Conclusion and Future Work.....	26
References.....	27



1 . Introduction

1.1 Background and Motivation

In daily life, people experience a wide range of emotional states such as happiness, sadness, anger, fear, stress, and anxiety. While some emotions, such as happiness or anger, are often easy to observe through facial expressions, stress and anxiety are usually more subtle and difficult to detect. These emotional states may appear through small facial changes, brief tension, or expressions that overlap with sadness or fear.

Facial expressions can provide useful clues about a person's emotional state. For this reason, computer vision techniques have been used to analyze facial features and support emotion recognition tasks. Many existing emotion recognition systems focus on identifying basic emotions such as happiness, sadness, anger, and fear. However, these systems are not always sufficient for detecting deeper emotional states such as stress and anxiety, which may require more careful analysis.

The need for stress detection systems has become increasingly important because stress and anxiety can affect mental health, learning performance, and daily life. Such systems can support psychologists, educators, and wellness applications by providing an additional tool to highlight possible signs of stress. The goal is not to replace professional diagnosis, but to assist in early observation and support decision-making.

Therefore, this project focuses on developing a computer vision based stress detection system using facial video analysis. The system uses video frames extracted from the RAVDESS dataset and applies a VGG16 transfer learning model to classify facial expressions into two categories: Stressed and Not Stressed. This approach aims to use facial expression changes across multiple frames rather than relying only on a single static image.



1.2 Problem Statement

Most facial emotion recognition systems are designed to recognize clear and basic emotions such as happiness, sadness, anger, fear, surprise, and neutral. However, stress and anxiety are more complex emotional states that may not appear as strong or obvious expressions. They may also share visual characteristics with sadness, fear, or neutral expressions, which makes automatic classification more difficult.

Another limitation is that many existing approaches rely on static facial images. A single image may capture a neutral or transitional moment rather than the most expressive part of an emotional state. Therefore, static image analysis may miss short facial changes, temporary tension, or expression shifts that occur across time.

1.3 Research Objectives

This project aims to build a system that can detect signs of stress from facial expressions using computer vision and deep learning.

The specific objectives are:

- To use facial video data for stress-related classification instead of relying only on static images.
- To reorganize emotional labels into two binary classes: Stressed and Not Stressed.
- To extract multiple frames from each video in order to capture facial expression variation over time.
- To investigate the effectiveness of VGG16 transfer learning as a feature extraction backbone for binary stress classification.
- To train and evaluate the proposed model using standard classification metrics, including Accuracy, Precision, Recall, F1-score, Confusion Matrix, ROC Curve, and ROC-AUC.



2. Related Work

Research in facial emotion recognition has developed significantly with the availability of benchmark datasets and deep learning models. Early and widely used approaches relied mainly on static facial images. One of the most common datasets in this field is FER-2013, which was introduced as part of the Challenges in Representation Learning and contains facial images labeled with basic emotion categories [1]. FER-2013 has been widely used for training and evaluating facial expression recognition systems, but it is mainly designed for general emotion recognition rather than direct stress detection.

Several studies have investigated stress detection using facial expressions. Almeida and Rodrigues proposed a facial expression recognition system for stress detection using deep learning and transfer learning models, including VGG16, VGG19, and Inception-ResNetV2. Since publicly available datasets directly labeled as stress/non-stress are limited, their study associated selected negative emotions, such as anger, disgust, and fear, with stress-related states. Their best model, VGG16, achieved 92.1% accuracy in binary stress/non-stress classification [2]. Similarly, Anusha et al. proposed a stress level classification system using facial images from the FER-2013 dataset. Their study compared a CNN-based model with a hybrid model based on residual networks and backtracking, where the hybrid model achieved higher test accuracy. This supports the use of facial features for stress-related classification [3]. However, both studies rely mainly on static facial images, which may not capture facial expression changes that occur over

CNN-based and transfer learning approaches have also been widely explored in facial emotion recognition. Bai studied the application of VGG16 in facial emotion recognition and showed its effectiveness as a feature extraction architecture [4]. Ge et al. reviewed deep learning methods for facial expression recognition and highlighted the importance of CNN-based models in automatically learning



discriminative facial features [5]. Nathani compared CNN and modified VGG16 models for emotion recognition using FER-2013 and AffectNet, showing that VGG16-based models can outperform simpler CNN baselines, while also noting that performance may decrease on more diverse datasets [6]. These studies support the use of VGG16 transfer learning in this project while also emphasizing the challenge of generalization.

Although static image-based systems are effective and computationally efficient, they capture only one moment of a facial expression. This limitation is important for stress and anxiety detection because stress-related cues may appear as brief facial tension, subtle expression changes, or transitions across time. Video-based emotion recognition addresses this limitation by analyzing facial expressions across multiple frames. Noroozi et al. studied emotion recognition in video clips using audio-visual information. Although their work included audio features, it is relevant to the current project because it highlights the importance of temporal visual information in video-based emotion analysis [9]

The RAVDESS dataset is widely used for audio-visual emotion recognition and contains emotional speech and song recordings from human actors under controlled conditions [7]. Luna-Jiménez et al. used RAVDESS for multimodal emotion recognition with transfer learning, demonstrating the suitability of the dataset for audio-visual emotion analysis [8]. However, RAVDESS contains acted emotional expressions rather than spontaneous real-world stress responses. Canedo and Neves reviewed facial expression recognition methods using computer vision and highlighted the development from traditional handcrafted approaches to deep learning-based models. Their review also emphasized that real-world facial expression recognition remains challenging due to variations in lighting, pose, occlusion, and dataset conditions [10]. This supports the need to carefully interpret model performance when using controlled datasets such as RAVDESS.

Based on these studies, the current project combines CNN-based facial feature extraction with video-frame-based facial analysis. Instead of relying on a single



static image, the proposed system extracts 10 frames from each RAVDESS video and uses a VGG16 transfer learning model to classify facial frames into Stressed and Not Stressed categories. This approach aims to capture richer facial information than static image methods while remaining computationally feasible.

2.1 Research Gap

Although previous studies achieved promising results in facial emotion recognition and stress-related classification, several gaps remain that motivate the design of this project.

First, many previous studies rely mainly on static image datasets such as FER2013. Static images are easier to process and require less computational cost, but they capture only one moment of a facial expression. This may cause the model to miss temporal facial changes such as brief tension, eye movement, or expression transitions. To address this gap, the current project uses multiple frames extracted from video clips instead of relying on a single static image.

Second, most available facial expression datasets are designed for general emotion recognition rather than direct stress detection. Because stress is not usually provided as a separate label, researchers often approximate stress by grouping related negative emotions into a broader stress-related category. In this project, sad, angry, and fearful expressions are mapped to the Stressed class, while calm and happy expressions are mapped to the Not Stressed class. This mapping follows the idea that negative emotional expressions can serve as visual proxies for stress-related states in facial expression analysis [2], [3].

Third, video-based analysis is more computationally demanding than imagebased analysis because it requires processing multiple frames from each video.



Instead of using the full video sequence, this project extracts 10 frames from each RAVDESS video. This design provides more facial variation than a single image while keeping the training process feasible on Kaggle GPU resources.

Finally, RAVDESS contains acted emotional expressions recorded under controlled conditions [7]. Although this makes the dataset clean and suitable for model training, it may not fully represent spontaneous stress in real-world environments. Therefore, real-world factors such as lighting, head pose, occlusion, and individual differences remain important limitations [10].

2.2 DataSets

- **Dataset Name:**

DAiSEE (Dataset for Affective States in E-Environments)

- **Dataset Sources Used in the Project:**

- Official Dataset Source:

[DAiSEE Official Dataset Page](#)

- Kaggle Dataset Source (used in the CNN+LSTM experiment):

[DAiSEE Kaggle Dataset](#)

- **Dataset Type:**

Dataset for affective state recognition

- **Dataset Content:**

The dataset contains short video clips of participants recorded during online learning sessions. Each video is annotated with affective states including:

- Engagement
- Boredom
- Confusion
- Frustration



- **Stress Labels Used in the Experiments:**

The Frustration label was used as the stress indicator:

0 = Very Low Frustration

1 = Low Frustration

2 = High Frustration

3 = Very High Frustration

In the **CNN+LSTM experiment**, the labels were converted into binary classes as follows:

- 0 and 1 → Not Stressed
- 2 and 3 → Stressed

In the **MATLAB-based VGG16 experiment**, a different binary mapping was used:

- 0 → Not Stressed
- 1, 2, and 3 → Stressed

- **Image/Video Type:**

Colored RGB video frames

- **Frame Size Used in the Experiments:**

224 × 224 pixels was commonly used in the experiments.

- **Preprocessing Applied:**

The dataset was preprocessed based on the requirements of each experiment. In general, the videos were loaded, frames were extracted, facial information was prepared, and the data was converted into a suitable format for deep learning model training and evaluation.

- **Number of Frames Used Per Video:**

The number of extracted frames varied depending on the experiment. For example, the CNN+LSTM experiment used 10 frames per video.



- **Dataset Name:**

RAVDESS (Ryerson Audio-Visual Database of Emotional Speech and Song)

- **Dataset Source:**

[RAVDESS Kaggle](#)

[Dataset](#)

- **Dataset Type:**

Audio-visual emotion recognition dataset

- **Dataset Content:**

The dataset contains video recordings of professional actors performing emotional expressions.

- **Emotions Included in the Dataset:**

- Neutral
- Calm
- Happy
- Sad
- Angry
- Fearful
- Surprise
- Disgust

- **Stress Labels Used in the Experiments:**

The emotions were converted into binary stress classes:

- Sad
- Angry
- Fearful
- Stressed



- Calm
- Happy
- Not Stressed

Other emotions were excluded from the experiments.

- **Image/Video Type:**

Colored RGB video frames

- **Frame Sizes Used in the Experiments:**

- 224 × 224 pixels (VGG16-based experiments)
- 128 × 128 pixels (MobileNetV2 + GRU experiment)

- **Preprocessing Applied:**

The dataset was preprocessed differently depending on the experiment. In general, videos were loaded using OpenCV, frames were extracted, resized, normalized, and prepared for deep learning model training and evaluation.

- **Number of Frames Used Per Video:**

- 10 frames per video (Final VGG16-based solution)
- 15 frames per video (MobileNetV2 + GRU experiment)

- **Total Videos Used:**

1920 videos



3 . Proposed Methodology & Implementation

3.1 CNN+LSTM Experiment

The first experiment was implemented using a CNN+LSTM architecture on the DAiSEE dataset for stress detection. The experiment focused on binary stress classification using facial expression analysis from video data.

During preprocessing, video frames were extracted using OpenCV, resized, normalized, and grouped into temporal sequences before training. Unlike the proposed VGG16 solution, this experiment focused on sequence-based learning by analyzing changes between consecutive video frames rather than classifying frames individually.

The architecture combined a pretrained ResNet18 convolutional neural network with an LSTM layer. ResNet18 was used to extract spatial facial features from each frame, while the LSTM layer learned temporal facial expression changes across the video sequence. Additional layers such as Dropout and fully connected classification layers were used to reduce overfitting and perform binary stress classification.

The model was trained using the Adam optimizer and BCEWithLogitsLoss loss function. Separate training, validation, and testing sets were used during model development. The evaluation process included accuracy, precision, recall, F1-score, confusion matrix, ROC curve, and ROC-AUC score to measure classification performance.

The experiment demonstrated that combining convolutional feature extraction with temporal learning could capture stress-related facial patterns from video sequences. However, the final classification performance was weaker and less stable compared to the proposed VGG16-based solution.

- **Why CNN+LSTM Was Not Selected as the Final Solution?**

Although the CNN+LSTM experiment was useful as a baseline model, it was not selected as the final solution because the architecture was more computationally complex than the proposed VGG16-based approach. The implementation required sequence-based video processing, frame extraction, face detection, ResNet18 feature extraction, and LSTM temporal learning, which



increased training complexity and computational cost.

In addition, the model achieved lower and less stable classification performance compared to the proposed solution. The experiment also required more preprocessing and temporal sequence handling, making the pipeline more difficult to optimize and deploy.

Compared with this experiment, the proposed VGG16 solution provided a simpler and more efficient workflow while achieving stronger overall performance and better implementation stability.

Therefore, the VGG16-based model was selected as the final proposed solution because it provided a better balance between performance, simplicity, and implementation efficiency.

3.2 MobileNetV2 + GRU (kaggle) Experiment

Another experiment was implemented using a MobileNetV2 + GRU architecture on the RAVDESS dataset to perform stress detection from facial video sequences. Similar to the proposed solution, the experiment converted emotional labels into binary stress categories by grouping stress-related emotions as stressed and positive emotions as not stressed.

During preprocessing, video frames were extracted using OpenCV and resized before normalization. Unlike the proposed VGG16 solution, this experiment processed a larger sequence of video frames in order to capture more temporal facial changes across the video. A custom TensorFlow data generator was also implemented to load and preprocess videos dynamically during training.

The architecture combined a pretrained MobileNetV2 convolutional neural network with a GRU layer. MobileNetV2 was used for lightweight spatial feature extraction from facial frames, while the GRU layer learned temporal relationships between consecutive frames. TimeDistributed processing was applied so that feature extraction could be performed independently on each frame before temporal learning.



Additional layers including GlobalAveragePooling2D, Dropout, and Dense classification layers were added to improve feature representation and reduce overfitting. EarlyStopping and ModelCheckpoint were also applied during training to improve model stability and save the best-performing model weights.

The experiment achieved strong classification performance and demonstrated that combining convolutional feature extraction with temporal learning can effectively detect stress-related facial patterns. Confusion matrices, classification reports, and training curves showed that the model was capable of distinguishing between stressed and not stressed samples with high accuracy.

- **Why MobileNetV2 + GRU Was Not Selected as the Final Solution?**

Although the MobileNetV2 + GRU experiment achieved strong results, it was not selected as the final solution because the implementation was more complex than the proposed VGG16-based approach. The model required sequence-based video processing, TimeDistributed feature extraction, and GRU temporal learning, which increased computational cost and implementation complexity.

Compared to this experiment, the proposed VGG16 solution provided a simpler and more efficient pipeline while still achieving strong performance. The VGG16based implementation was easier to train, required a less complex architecture, and was more suitable for stable deployment and real-time integration. Therefore, the VGG16-based model was selected as the final proposed solution due to its balance between performance, simplicity, and implementation efficiency.

3.3 Real-Time VGG16 Colab Experiment

This experiment was implemented to test the trained VGG16 stress detection model in a real-time environment using Google Colab. Unlike the proposed solution, which focused on training and evaluating the model using dataset videos, this experiment focused on testing the model with live webcam images.

The code rebuilt the same VGG16-based architecture used for binary stress classification. The model included a VGG16 base model, Flatten layer, Dense layer, Dropout layer, and Sigmoid output layer for binary classification between stress and not stress.



Instead of training the model again, the experiment loaded the saved weights from the trained VGG16 model. This allowed the code to directly use the pretrained model for webcam-based prediction.

The experiment used JavaScript inside Google Colab to access the device webcam. When the user pressed the Capture button, a single image was captured from the webcam and saved for prediction. Therefore, the implementation operated as a shot-by-shot prediction system rather than continuous live video processing. After capturing the image, the base64 image data was decoded and stored as a JPEG file. The captured image was then read using OpenCV for preprocessing.

During preprocessing, the image was resized to 224×224 pixels to match the VGG16 input size. The pixel values were normalized by dividing them by 255, and the image was converted into batch format before being passed to the model.

The preprocessed image was then passed to the trained VGG16 model using `model.predict()`. The model generated a raw prediction score representing the probability of stress.

A threshold value was applied to convert the raw prediction score into a final classification label. If the prediction score exceeded the threshold, the image was classified as Stress. Otherwise, it was classified as Not Stress.

The final prediction label and confidence percentage were displayed directly on the captured image using OpenCV. Red text was used for Stress predictions, while green text was used for Not Stress predictions. The final annotated image was then displayed inside the Colab environment together with the raw prediction score.

The experiment successfully demonstrated that the trained VGG16 model could be integrated into a real-time prediction pipeline using webcam input. However, unlike the proposed solution, this experiment focused only on real-time prediction and model deployment rather than model training and evaluation on large datasets.



- **Why Real-Time VGG16 Colab Was Not Selected as the Final Solution?**

Although the implementation was functional, it was not selected as the final solution because it focused only on real-time testing rather than full model training and evaluation. The experiment depended on a single captured webcam image, so it did not analyze video sequences or evaluate performance using a complete testing dataset. In addition, the prediction stability was affected by external factors such as lighting conditions, camera quality, image angle, and background variations. Since the system processed only one captured image at a time, it was less reliable than a full video-based evaluation pipeline.

Therefore, this experiment was mainly used to demonstrate the practical usability of the trained VGG16 model in a real-time environment, while the proposed VGG16-based solution remained the final selected implementation for training, evaluation, and overall system development.

3.4 MATLAB-Based VGG16 Experiment

Another experiment was implemented using MATLAB and a VGG16-based deep learning model on the DAiSEE dataset for stress detection. Similar to the proposed solution, this experiment focused on binary classification between stressed and not stressed facial expressions using transfer learning.

The DAiSEE dataset was used because it contains video clips annotated with affective states including engagement, boredom, confusion, and frustration. In this experiment, the Frustration label was selected as the stress indicator and converted into binary stress classes. A frustration score of 0 was labeled as Not Stressed, while scores 1, 2, and 3 were labeled as Stressed.

The first stage involved dataset loading and verification. Video files and label files were loaded from the dataset directory to ensure that all samples and annotations were correctly matched before preprocessing. In this experiment, a subset of 300 training videos was processed due to computational and time constraints.



During preprocessing, the videos were processed frame by frame using MATLAB. One frame was sampled every 15 frames from each video sequence, and facial regions were detected using MATLAB's built-in face detector. The largest

detected face was selected from each frame and resized to 224×224 pixels to match the VGG16 input size.

A maximum of 10 face images was extracted from each video and stored in separate folders representing stressed and not stressed classes. This preprocessing approach converted raw video data into structured image data

suitable for deep learning training.

The experiment used a pretrained VGG16 convolutional neural network through transfer learning. The extracted facial images were loaded using MATLAB image datastore functionality, and the dataset was divided into training and testing sets before model training. The dataset was split into 80% training and 20% testing using MATLAB's `splitEachLabel` function.

The original VGG16 classification layers were replaced with custom layers for binary stress classification. The final architecture included a VGG16 feature extractor followed by fully connected classification layers for stressed and not stressed prediction.

The model was trained using stochastic gradient descent with momentum (SGDM). Training parameters such as batch size, learning rate, and number of epochs were configured within the MATLAB training environment. After training, the model was evaluated using the testing dataset. The evaluation process included classification accuracy and confusion matrix analysis to measure the model's ability to distinguish between stressed and not stressed facial expressions. The experiment demonstrated that transfer learning with VGG16 could successfully classify stress-related facial patterns using MATLAB-based preprocessing and training workflows. The confusion matrix showed that the model was capable of correctly classifying most stress-related samples.

- **Why MATLAB-Based VGG16 Was Not Selected as the Final Solution?**

Although the MATLAB-based VGG16 experiment achieved good classification performance, it was not selected as the final solution because the implementation



was less flexible compared to the proposed Python and TensorFlow-based approach. The MATLAB workflow required separate preprocessing and dataset organization steps before training, which increased implementation complexity. In addition, integration with real-time deployment and external deep learning libraries was more limited compared to the Kaggle and TensorFlow environments used in the proposed solution.

Compared with this experiment, the final VGG16-based implementation provided a more efficient workflow, easier customization, and better compatibility with real-time testing and deployment environments.

Therefore, the Python-based VGG16 implementation was selected as the final proposed solution because it provided stronger flexibility, easier experimentation, and more practical deployment capabilities.

3.5 Final VGG16-Based Solution

This section explains the methodology and implementation of the proposed stress detection system. The system uses deep learning and computer vision techniques to classify facial video frames into stressed and not stressed categories.

The implementation was developed using Python and TensorFlow/Keras in the Kaggle environment. The RAVDESS dataset was used for training and evaluation. The proposed system consists of several stages including dataset preparation, preprocessing, feature extraction, model training, and performance evaluation.



- **System Flow Diagram**

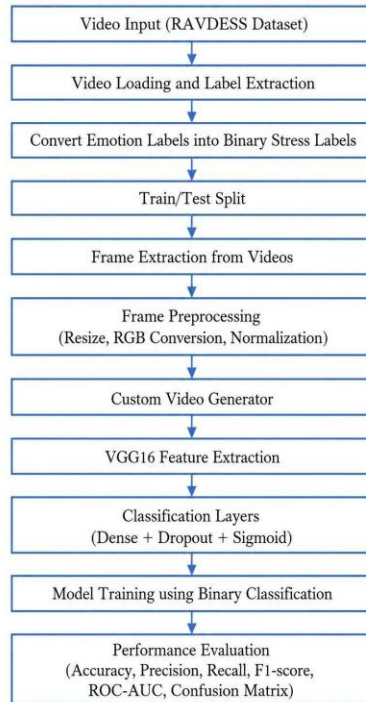


Figure 1: System Flow Diagram

- **Dataset Preparation**

The RAVDESS dataset was used in this project. The dataset contains video recordings of actors performing different emotional expressions.

Video files were loaded from the dataset directory using Python. The emotion labels were converted into binary stress classes based on the emotion ID extracted from the filename.

The emotions angry, fearful, and sad (labels 4, 5, and 6) were classified as stressed, while calm and happy (labels 2 and 3) were classified as not stressed. Other emotions, including neutral, disgust, and surprised expressions, were excluded because they were not part of the selected binary classification categories for this study.

After label generation, the dataset was divided into training and testing sets using stratified splitting to preserve class distribution.

The final dataset split included:

- ❖ 1536 training videos
- ❖ 384 testing videos



- **Video Preprocessing**

Video preprocessing was performed before training the model. Each video was processed using OpenCV.

First, up to 10 frames were extracted from each video. The extracted frames were resized to 224×224 pixels to match the VGG16 input size. After resizing, frames were converted from BGR to RGB format.

Finally, pixel values were normalized by dividing them by 255 to improve training stability and model performance.

- **Custom Video Generator**

A custom video generator was implemented using TensorFlow Sequence. The generator loads videos in batches during training instead of loading the entire dataset into memory.

The generator is responsible for:

- ❖ Reading video files
- ❖ Extracting frames
- ❖ Applying preprocessing
- ❖ Returning batches of frames and labels

Each video was converted into multiple frames, and the original video label was assigned to all extracted frames.

This approach improves memory efficiency and supports large video datasets.

- **VGG16-Based Model**

The proposed model is based on the VGG16 convolutional neural network pretrained on ImageNet.

The top classification layers of VGG16 were removed and replaced with custom layers for binary stress classification. The final architecture includes:

- ❖ VGG16 base model
- ❖ Flatten layer
- ❖ Dense layer with ReLU activation
- ❖ Dropout layer
- ❖ Sigmoid output layer

The pretrained VGG16 layers were frozen during training to reduce computational cost and prevent overfitting.



- **Model Training**

The model was compiled using:

- ❖ Adam optimizer
- ❖ Binary Crossentropy loss function
- ❖ Accuracy metric

Training was performed for up to 20 epochs using the training generator.

Early stopping was applied to monitor validation loss and restore the best model weights when performance stopped improving. The testing generator was also used as validation data during training.

This helped reduce overfitting and improve generalization.

- **Model Evaluation**

The trained model was evaluated using the testing dataset.

Several evaluation metrics were implemented in the evaluation stage, including:

- ❖ Accuracy
- ❖ Precision
- ❖ Recall
- ❖ F1-score
- ❖ ROC-AUC score

A confusion matrix was generated to visualize correct and incorrect predictions for both classes. In addition, an ROC curve was plotted to measure the model's classification capability and overall classification performance.



4. Experimental Setup

This section describes how the experiments in our stress detection project were actually carried out.

The goal is to give a clear picture of the environment we worked in, the tools we used, and the exact setup that allowed us to train and evaluate our model reliably.

- **Hardware and Software Environment**

All experiments were implemented using a combination of local development tools and cloud resources. We relied mainly on:

- Windows 10 (64-bit) as the operating system
- PyCharm 2024.3 for local development
- Kaggle Notebook for GPU-accelerated training
- Python 3.10 as the main programming language
- TensorFlow 2.10 and Keras for deep learning
- OpenCV, NumPy, Matplotlib, Scikit-learn for preprocessing and evaluation

Using Kaggle's GPU significantly reduced training time and made it easier to experiment with different configurations.

- **Dataset Configuration**

The experiments were based on the RAVDESS video dataset, which contains recordings of actors expressing different emotions.

For our binary classification task (stressed vs. not stressed), we reorganized the dataset as follows:

- Stressed: Angry, Fearful, Sad
- Not Stressed: Calm, Happy
- Excluded: Neutral, Disgust, Surprise

After filtering and labeling, the dataset was split into:

- 1536 training videos
- 384 testing videos

From each video, up to 10 frames were extracted. Every frame was:

- Resized to 224×224
- Converted to RGB
- Normalized to the range $[0, 1]$

This ensured that all inputs matched the VGG16 model requirements.



- **Training Configuration**

The model was trained using a VGG16-based architecture with custom classification layers. To keep training stable and efficient, we used the following settings:

- Optimizer: Adam
- Learning Rate: 0.0001
- Loss Function: Binary Crossentropy
- Batch Size: 32
- Epochs: 20
- Validation Split: 20%
- Early Stopping: Enabled to prevent overfitting

A custom video generator handled loading and preprocessing videos on the fly, which helped avoid memory issues and made training smoother.

- **Evaluation Metrics**

To measure how well the model performed, we used a set of widely accepted metrics:

- Accuracy
- Precision
- Recall
- F1-Score
- ROC-AUC
- Confusion Matrix

These metrics allowed us to evaluate not only the overall performance but also how well the model distinguished between stressed and non-stressed expressions.

- **Summary**

Overall, this experimental setup provided a stable and reproducible environment for training and evaluating our stress-detection model. The combination of GPU-accelerated training, structured preprocessing, and comprehensive evaluation metrics ensured that the results presented in the next section are both reliable and meaningful.

5. Experimental Results and Discussion

• Results & Analysis

After conducting a series of experiments on various neural network architectures, the VGG16 model (trained on the RAVDESS dataset) demonstrated superior performance in terms of accuracy and computational efficiency. The model achieved an overall accuracy of 95.52%, as shown in the performance metrics in **Figure 4**, surpassing MobileNetV2 + GRU and other tested methodologies.

Comparative analysis revealed that VGG16 possesses a remarkable ability to extract vital emotional patterns using only 10 frames and fewer epochs. This significantly reduces time and computational resource consumption compared to MobileNetV2 + GRU, which required more extensive training and data density to reach similar performance levels. Examination of the Confusion Matrix (**Figure 2**) and the ROC Curve (**Figure 3**), along with metrics such as Precision (0.966)

and Recall (0.958) from Figure 4, indicates that the model has high sensitivity in distinguishing stress from other emotions. The ROC curve specifically demonstrates an AUC of 0.9815 (**Figure 3**), confirming the model's robustness, reliability, and its ability to generalize patterns rather than merely memorizing data.

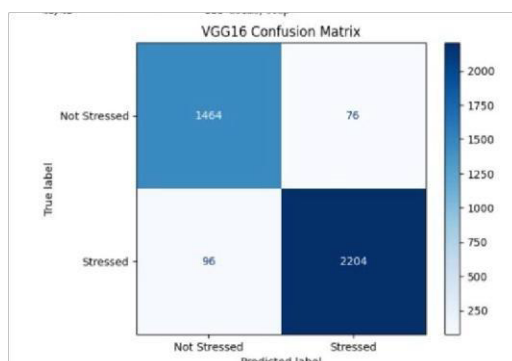


Figure 2 :Confusion Matrix for VGG16 Model

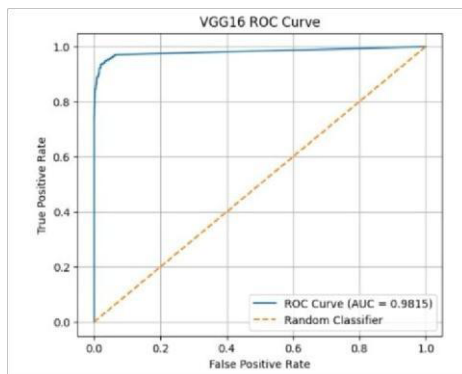


Figure 3: ROC Curve for VGG16 Model

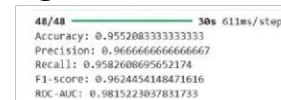


Figure 4: Output of VGG16 Model

● Discussion

These results validate the feasibility of using deep learning models to classify anxiety and stress, providing decision-making support across various contexts. The application of this project is not limited to psychiatric clinics; it extends to broader fields such as monitoring professional and educational performance. While external factors (such as lighting, sudden movement, or poorly timed frame capture) can affect the model, these gaps can be mitigated through intensified training on more diverse and precise datasets. This opens the door to transforming this model into an intelligent analytical tool for real-world environments.



● Error Analysis

Despite the advanced performance levels achieved, a detailed analysis revealed certain challenges, specifically regarding the misclassification of stressed individuals as calm and vice versa. These errors are attributed to several potential technical factors:

- Environmental and Kinetic Factors: Rapid movement during frame acquisition.
- Pattern Similarity: The model occasionally struggles with overlapping features, leading to the misclassification of "Stressed" as "Not Stressed" and vice versa. This indicates a crossover in interpreting subtle, similar expressions, as evidenced by the Confusion Matrix in **Figure 2**, which shows 76 instances where "Not Stressed" was predicted as "Stressed" and 96 instances where "Stressed" was incorrectly labeled as "Not Stressed".

6. Identified Limitation

Our System faces some challenges that affect its performance in real world situations. The most common issue is that the model is very sensitive to lighting and camera quality which can make the predictions less accurate during a live session. We also noticed that the real time results are not always stable due to background noise and hardware limits. Another challenge is the dataset we used; since it features professional actors in a controlled studio, it might not perfectly match how people show stress naturally in daily life. Finally, if a person moves their head a lot or covers part of their face, the model struggles to detect the facial signs properly.



7. Conclusion and Future Work

To conclude, this project successfully created a system to detect stress using the VGGNet model. We chose this model because it was the most efficient and gave us great results without needing too many training cycles. Our testing showed that the model is very effective, reaching a validation accuracy of 95.52% on the RAVDESS dataset. By using video data we were able to help the model learn how facial expressions change over time which is more reliable than looking at a single photo. This project proves that we can use computer vision to help mental health professionals by identifying signs of stress.

In the future, we plan to make the system better and more reliable for everyday use by following a multimodal approach that incorporates full body language and kinetic gestures beyond facial expressions alone. We also hope to improve the model's precision through domain adaptation by training it on specialized clinical datasets focused specifically on stress disorders. Another goal is to optimize preprocessing by integrating advanced algorithms for lighting correction and reducing the impact of visual obstructions to ensure stable performance in variable conditions. Furthermore, we aim to use data augmentation to train the model on low resolution data for better generalization. Eventually, we want to turn this into a fully automatic system that can work continuously in places like clinics, schools, or offices to help people manage their emotional state.



References

- [1] I. J. Goodfellow et al., “Challenges in representation learning: A report on three machine learning contests,” arXiv preprint arXiv:1307.0414, 2013. [Online]. Available: <https://arxiv.org/abs/1307.0414>
- [2] J. Almeida and F. Rodrigues, “Facial expression recognition system for stress detection with deep learning,” in Proc. International Conference on Computer Vision Theory and Applications (VISAPP), 2021. [Online]. Available: <https://www.scitepress.org/PublishedPapers/2021/104742/>
- [3] J. A. Anusha et al., “Stress level classification using facial images,” in Proc. International Conference on Signal, Image Processing, Communication and Automation, Atlantis Press, 2024. [Online]. Available: <https://www.atlantis-press.com/proceedings/sipcov-23/126003945>
- [4] Y. Bai, “Application and analysis of the VGG16 model in facial emotion recognition,” in Proc. International Conference on Computer Vision, Image and Deep Learning, 2023. [Online]. Available: <https://www.scitepress.org/Papers/2023/127998/>
- [5] H. Ge et al., “Facial expression recognition based on deep learning,” Computer Methods and Programs in Biomedicine, vol. 215, p. 106621, 2022, doi: 10.1016/j.cmpb.2022.106621.
- [6] S. Nathani, “A comparative study of transfer learning for emotion recognition using CNN and modified VGG16 models,” arXiv preprint arXiv:2407.14576, 2024. [Online]. Available: <https://arxiv.org/abs/2407.14576>
- [7] S. R. Livingstone and F. A. Russo, “The Ryerson Audio-Visual Database of Emotional Speech and Song (RAVDESS): A dynamic, multimodal set of facial and vocal expressions in North American English,” PLOS ONE, vol. 13, no. 5, p. e0196391, 2018, doi: 10.1371/journal.pone.0196391.



- [8] C. Luna-Jiménez et al., “Multimodal emotion recognition on RAVDESS dataset using transfer learning,” *Sensors*, vol. 21, no. 22, p. 7665, 2021, doi: 10.3390/s21227665.
- [9] F. Noroozi, M. Marjanovic, A. Njegus, S. Escalera, and G. Anbarjafari, “Audio-visual emotion recognition in video clips,” *IEEE Transactions on Affective Computing*, vol. 10, no. 1, pp. 60–75, 2019, doi: 10.1109/TAFFC.2017.2713783.
- [10] S. Canedo and A. J. R. Neves, “Facial expression recognition using computer vision: A systematic review,” *Applied Sciences*, vol. 9, no. 21, p. 4678, 2019, doi: 10.3390/app9214678.
- [11] A. Gupta, A. D’Cunha, K. Awasthi, and V. Balasubramanian, “DAiSEE: Towards user engagement recognition in the wild,” IIT Hyderabad. [Online]. Available: <https://people.iith.ac.in/vineethnb/resources/daisee/index.html>